ARKIV GLOBAL TOUR STOP · BEST INDIAN USE CASE TRACK

Bhaiya Please

"Bhaiya, please le aana if possible"

Someone is already coming to your building.
Ask them to pick up the one thing you need on the way.

Idea and build proposal · v2 · 16 October 2026 · Team of two

In one paragraph

Every day lakhs of delivery riders from Swiggy, Zomato, Blinkit, Zepto and Instamart ride into the same apartment societies. Each of them passes small shops that are on none of those apps: the kirana, the chemist, the bakery at the corner. **Bhaiya Please** is an open, app-neutral board on Arkiv. Anyone already heading to a building can announce a short-lived *run*: a platform rider, a neighbour, or a student coming home. Residents of that building attach a small request from a shop on the runner's route. After the handoff at the gate, the resident issues a signed receipt and **transfers it to the runner's wallet**. Over time those receipts become a **reputation the runner owns**, which anyone can verify without trusting us, and which no platform can delete.

Question	Our answer
Is the idea good?	Yes. The problem is real and specific to India, the behaviour already happens informally, and no company owns the layer that works across all the platforms.
Does it fit the India track?	Yes. The track asks for a real application people in India can use, with no CROPS pillar required. Judges score who in India uses it and what changes for them.
Why can't this be WhatsApp or Firebase?	On Firebase, reputation is a *feature of our app*. On Arkiv it is an *asset of the runner*: held in their wallet, signed by the residents they helped, and checkable by any other app. Section 4 sets out the full argument.
Biggest risk	Time. Submissions close **18 Oct, 23:59 London time (04:29 IST on 19 Oct)**. The scope in section 10 is cut to fit.

What changed in v2: verification by attestation, receipts transferred to the runner, complaints that the runner cannot delete, open runners in reputation tiers, and a schema and code sketch corrected against the Arkiv SDK 0.8 docs.

1. The problem

- **Someone is already on the road to your building.** At peak hours a mid-sized society sees a rider arrive every few minutes, and most of them pass the local market on the way in. Neighbours come home from work the same way.
- **Your neighbourhood shops are invisible to the apps.** The kirana, the chemist and the bakery are not on Blinkit or Zomato. They stock the specific brand, the loose item, or the thing the dark store ran out of.
- **There is no trusted way to connect the two.** Asking a stranger to buy something means guessing who they are, whether they will come back, and how to pay.

A resident who wants one packet of bread walks out, places a ₹200 minimum order with delivery fees, or does without. A runner who could earn ₹20 to ₹40 for a short stop has no way to offer it.

Why the platforms will not build this

Each app sees only its own riders and catalogue. Swiggy will not show a Zomato rider to its users, and none of them will send traffic to a shop that pays them no commission. The useful version has to be **neutral**: any runner, any shop, any resident. A neutral layer cannot be owned by one of the players.

Reputation that disappears

A rider with 4,000 five-star deliveries on one app starts from zero on another, and loses everything if the account is deactivated, often with no appeal. Gig-worker unions in India have raised opaque deactivation as a core grievance. Bhaiya Please gives runners a small but real record that sits in **their own wallet**, is signed by the people they helped, and works wherever they go.

2. Who in India uses it

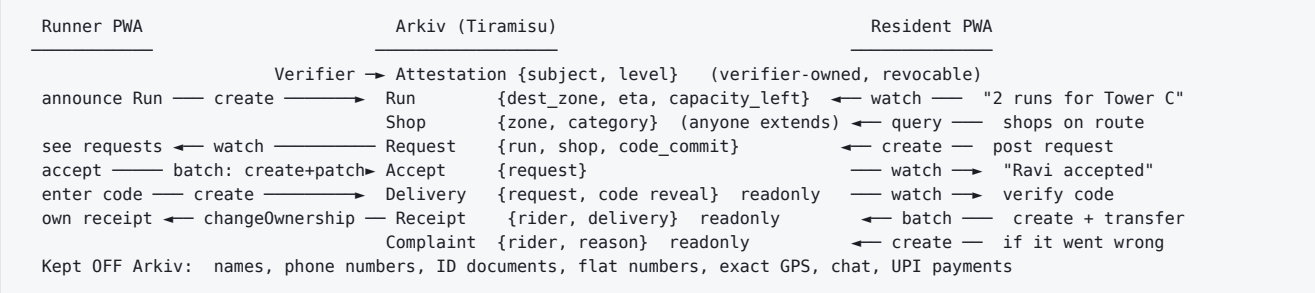
User	Today	With Bhaiya Please
Resident of a gated society or apartment in Bengaluru, Gurugram, Pune, Hyderabad or Mumbai	Walks out for one item, or pays delivery fees and a minimum order.	Sees who is arriving in the next 15 minutes, asks for one item from a named local shop, and collects it at the gate.
Platform rider (Swiggy, Zomato, Blinkit, Zepto), mostly on the way back from a drop	Paid only per platform order, with nothing for the return leg. Reputation is locked inside each app.	Earns a tip for a short stop on a trip they are already making, and builds a portable, verifiable record.
Independent runner: a neighbour, a student, someone coming home from work	Favours happen in WhatsApp groups with no structure, no record and phone numbers on show.	Can run once verified. Limits grow with reputation (section 6), so new runners start small.
Neighbourhood shop (kirana, chemist, bakery, stationery)	Not on quick-commerce, so it loses sales to dark stores.	Listed by the community for free. It stays on the map while neighbours keep confirming it is open.
Verifier: a society's resident welfare association, a gig-workers' union, or for the demo, our own verifier wallet	Gate guards check IDs by hand and keep no shared record.	Checks a runner's ID once, in person, and issues an attestation that every society can see. No ID data goes on-chain.

> **8:40 pm, Tower C, a society in Whitefield, Bengaluru.** Priya runs out of curd. Two runs are live for her building: a Zepto rider arriving in 6 minutes and Ravi, a Swiggy rider on his way back, arriving in 11. Ravi is *ID-checked by the Whitefield gig-workers' union* and holds 212 receipts from 140 different residents. Priya requests "1 Nandini curd 500 g, max ₹40, tip ₹25" from Lakshmi Stores. Ravi accepts and reaches the gate. Priya shows him her handoff QR code and pays by UPI. Ravi's app records the delivery. Priya's app checks it, issues a receipt, and transfers it to Ravi's wallet. He now holds 213, and they stay his even if he changes apps tomorrow.

All names, places and numbers are illustrative. The demo uses synthetic data only, as the rules require.

3. How it works

#	Who	What happens	Arkiv entity and owner
0	Verifier	Checks the runner's ID in person, then issues an attestation about the runner's wallet. It contains no name and no ID number.	**Attestation**, owned by the verifier, who can revoke it
1	Community	Lists a local shop. Any neighbour can tap "still open" to extend its life.	**Shop**, extendable by anyone
2	Runner	Announces a run: destination zone, ETA, coming-from zone and capacity.	**Run**, owned by the runner
3	Resident	Picks a shop on the runner's route and posts items, a price cap and a tip. The app commits to a secret handoff code.	**Request**, owned by the resident
4	Runner	Accepts. One atomic batch creates the Accept and reduces the run's remaining capacity.	**Accept** plus a patch to the Run
5	Runner	At the gate, scans the resident's handoff QR code. Payment goes over UPI, off-chain.	**Delivery**, read-only, owned by the runner
6	Resident	The app checks the code. If all went well, it issues a read-only receipt and transfers it to the runner. If not, it files a complaint that the resident keeps.	**Receipt** (moves to the runner) or **Complaint** (stays with the resident)
7	Anyone	Opens a runner's profile: attestations, receipts from distinct residents, and recent complaints.	Queries by owner and creator



The handoff code: proof of a meeting without personal data

When posting a request, the resident's app generates a random 4-digit code and a 16-byte salt, and writes only `keccak256(code || salt || request_key)` into the Request. At the gate the resident's phone shows a QR code holding the code and salt, and the runner scans it. The 4-digit code is also shown so the two can confirm it aloud. The runner's Delivery reveals both values, and anyone can check them against the hash. A Delivery therefore proves the two people met, without storing who they are. Because of the secret salt, a runner cannot brute-force the 10,000 possible codes to fake a delivery without meeting.

4. Why Arkiv, and not WhatsApp or Firebase

The rubric gives full marks only when the project would fall apart without the Arkiv property it relies on. These are the two alternatives a judge will reach for first.

Requirement	WhatsApp group	Firebase run by us	Arkiv
Runner can find residents across many societies	No. A runner is not in every society's group, and residents won't add strangers.	Yes	Yes
Strangers never see each other's phone numbers	No. Every member sees every number.	Yes	Yes. Pseudonymous wallets, and the meeting point is the gate.
Runs disappear when they stop being true	No. Messages linger and clutter the group.	Only with a cleanup job we run	Yes. Built-in expiry, no job to run.
Nobody can edit or delete a review after the fact	Messages can be deleted	No. The operator can edit any row.	Yes. Read-only entities, and $creator cannot be changed.
Reputation survives if the app shuts down	No	No. It dies with our database.	Yes. Held in the runner's wallet and archived to Ethereum.
Another app can verify a runner without asking us	No	No. Only through our API, and only if we allow it.	Yes. Anyone can query receipts and attestations, and check the creator.

The argument in one line: on Firebase, reputation is a feature of our app. On Arkiv, reputation is an asset of the runner. A housing society's gate app, a workers' union, a microlender, or a platform hiring the runner can all read and verify the same receipts, with no API key and no permission from Bhaiya Please.

What we deliberately keep off Arkiv

Data	Why	Instead
Names, phone numbers, flat numbers, ID documents	Personal data. The rules forbid storing it on Arkiv, even hashed.	Never collected. The verifier sees the ID in person and records only that a check happened.
Exact location and GPS trail	Coarse zones are enough to match people, and precise tracking is a safety risk.	A geohash only (150 m to 1.2 km). Exact position stays on the device.
Payment	UPI is already trusted. Putting money on-chain adds risk and no benefit.	Direct UPI from resident to runner at the gate.
Chat and platform order details	Private, high-volume, and not ours to store.	Out of scope. Bhaiya Please never touches any delivery app's data.

5. How we use Arkiv: each feature does a visible job

The rubric judges how and why each feature is used, and counts features that are claimed but absent against the team. Every row below appears in the code and changes something on screen.

Feature (SDK 0.8)	Where	What the user sees
`readonly` + `changeOwnership`	The resident creates a read-only Receipt and transfers it to the runner in the same batch. $creator stays the resident.	The runner *holds* their reputation, every receipt shows who vouched for it, and nobody can edit it, including us.
Ownership asymmetry	Receipts are owned by the runner. Complaints stay owned by the resident. Attestations stay owned by the verifier.	Runners carry their good record anywhere but cannot delete a complaint. A verifier can revoke an attestation; the runner cannot fake one.
`permissionless-Extension`	Shop entities	Any neighbour can tap "still open". Closed shops quietly fade away. No moderator.
Expiry	Run (ETA + 10 min), Request, Accept, Attestation (180 days), Complaint (180 days)	Old runs never show up. Verification must be renewed. Complaints fade after six months.
`executeBatch` (atomic) + `predictEntityKeys`	Accept with capacity patch; Receipt create plus transfer	No booking beyond capacity. A receipt is never left half-issued.
`watchEntityEvents`	Run, Request, Accept and Delivery screens	Accepted and delivered states appear on the other phone within seconds.
Typed attributes (u64, `i32`, `addr`, key)	eta, capacity_left, rider, request	Range queries such as `eta > now`, and joins between entities by key.
`atBlock` **historical query**	Dispute view	"This was the runner's tier and record at the moment you accepted." Nobody can rewrite it.

Schema (draft for /arkiv/schema.md)

Every entity carries `project = "bhaiyaplease"` and `type`, as the Arkiv best-practices page advises. Arkiv is a shared public database, so this keeps our data separate from other projects.

Entity	Key attributes	Flags	Expiry	Owner / creator
Attestation	subject (addr), level (str: id_checked, resident, successor), prev (addr, successor only), at (u64)	readonly	180 days	Verifier / verifier
Shop	zone, zone5, category, name_tag	permissionless-Extension	30 days, extended by anyone	Lister / lister
Run	dest_zone, from_zone, eta (u64), capacity_left (i32), tier (i32)	none	ETA + 10 min	Runner / runner
Request	run (key), shop (key), dest_zone, max_price (i32), tip (i32), code_commit (bytes32)	readonly	Same as its run	Resident / resident
Accept	request (key), run (key), rider (addr)	none	Same as its run	Runner / runner
Delivery	request (key), rider (addr), resident (addr), zone; payload holds the code reveal	readonly	1 year	Runner / runner
Receipt	rider (addr), delivery (key), zone, at (u64)	readonly	1 year; the runner may extend	**Runner** / resident
Complaint	rider (addr), request (key), reason (str: no_show, wrong_item, rude), at (u64)	readonly	180 days	Resident / resident

Screen	Query (Arkiv query language)
Live runs to my building	`type = str('run') AND dest_zone = str($z) AND eta > u64($now) AND capacity_left > 0`
Shops on the runner's route	`type = str('shop') AND zone5 = str($from5) AND category = str('grocery')`
Requests on my run	`type = str('request') AND run = key($run)`
Runner's receipts	`type = str('receipt') AND $owner = addr($rider)`, then count distinct $creator in the app
Runner's verification	`type = str('attestation') AND subject = addr($rider) AND ($creator = addr($v1) OR $creator = addr($v2))`
Recent complaints	`type = str('complaint') AND rider = addr($rider) AND at > u64($now - 30d)`

6. Reputation and verification

Reputation rules

- **Count receipts, not stars.** A receipt only counts if its Delivery's code reveal matches the Request's commitment. The profile says "212 handoffs from 140 different residents" rather than "4.8 stars".
- **Count distinct residents** by $creator. Farming receipts from one's own second wallet gives a count of one.
- **Show a recent window** (last 30 days) next to the all-time count, so an old record can't hide a bad month.
- **Show complaints separately.** They are owned by the resident, so the runner cannot delete them. They expire after 180 days, so people can recover.
- **No score is stored on-chain.** We store raw receipts and compute the score in the app. Any other app can apply its own formula to the same data.
- **Key loss is survivable.** A runner who still has the old key moves receipts with changeOwnership. If the key is lost, a verifier issues a successor attestation linking the new wallet to the old one.

Verification without personal data

A verifier checks the ID in person and creates an Attestation about the runner's wallet, with a level and a 180-day expiry. **The verifier keeps ownership so they can revoke it** by deleting it. Receipts work the other way and move to the runner. More than one verifier can exist, such as a society's welfare association or a gig-workers' union, so there is no single gatekeeper. Residents choose which verifiers they trust, and the app shows who vouched. A future version could use Anon Aadhaar, a zero-knowledge proof of an Aadhaar holder that reveals no data. It is out of scope for this hackathon.

Open runners, in tiers

Anyone already heading to a building can run, not only platform riders. That widens supply and helps with cold start. One guardrail keeps the model honest: **a run must be a trip the runner is already making**. Once people make trips *for* the orders, it becomes a dedicated courier service, the model that failed on cost for Dunzo. Limits grow with trust, and each runner's tier is shown on their run card:

Tier	Requirement	Limits
New	Any wallet	Price cap ₹200, 1 request per run

Tier	Requirement	Limits
Verified	Attestation from a verifier the resident trusts	Price cap ₹500, 2 requests per run
Trusted	Verified, plus 25 or more receipts from 15 or more distinct residents, and fewer than 3 complaints in 30 days	Price cap ₹1,000, 3 requests per run

7. Trust, safety and honest limits

Risk	Mitigation
Runner takes money and disappears	No prepayment. The resident pays over UPI at the gate after seeing the item.
Resident doesn't show up after the runner has bought the item	The price cap limits the loss, and runners on lower tiers have lower caps. The runner files a no-show Complaint against the resident's wallet, which other runners can see.
Fake receipts from Sybil wallets	Only receipts from distinct residents count. The Trusted tier also needs a verifier attestation, which can't be farmed. Remaining limit: a determined attacker with several verified friends could inflate a record. Verifier accountability is the real defence.
Runner deletes bad feedback	Impossible. Complaints are owned by the resident. The runner holds only receipts, which are positive by design.
Slowing down a rider's platform order	The runner chooses, and the pitch centres on return trips after a drop. Only shops in the runner's from-zone are offered.
Platform rules	No integration with or scraping of any delivery app. It is a neutral board, like a society WhatsApp group but safer.
Restricted items	No tobacco, alcohol or prescription medicines. Categories: grocery, bakery, stationery, household, over-the-counter pharmacy.
Privacy	Coarse zones only. The meeting point is always the gate or lobby. No ID data, names or numbers ever go on-chain.

8. Fit with the hackathon and with past winners

Pattern in winners	Examples	Bhaiya Please
Expiry is the core of the product	Still Yours, Layak, Cortex	Runs last about 20 minutes, verification lasts 6 months, complaints fade after 6 months.
Signed claims from independent parties	Chorus, Strikethrough	Receipts are created by residents and held by runners. Attestations come from verifiers.
Renewal as proof of life	Tideland, Still Yours	Anyone can extend a shop listing, so the map stays truthful.
Marketplace or coordination board	MentorGraph ($4K), Shared Compute Lease Board	Runs as offers, requests as asks, receipts as completed bookings.

Criterion (25 pts each)	How we aim for a 5
Why India?	Named users and what changes for each. Plus 3 to 5 short interviews with real riders outside a dark store, quoted in the README and the demo.

Criterion (25 pts each)	How we aim for a 5
Why Arkiv?	The comparison with WhatsApp and Firebase (section 4), the "asset, not feature" argument, and an explicit off-Arkiv list.
How you use Arkiv	Every feature in section 5 has a visible effect, and the ownership choices are explained.
Friction report	Logged live as we build, with SDK version, the exact call, the error and a workaround.

9. Technical plan

Layer	Choice	Reason
Frontend	Vite + React PWA, Tailwind, mobile first	Runners and residents use Android phones, and a PWA installs without an app store.
Wallet	Browser burner key with export and import as a QR code. MetaMask optional.	No onboarding friction, and export covers key loss.
Arkiv	`@arkiv-network/sdk` 0.8.x on Tiramisu (chain 7738577), with a free access key	The default RPC is heavily rate-limited.
Verifier	A tiny verifier page: scan a runner's wallet QR code and issue an attestation	Demonstrates verification without storing ID data.
Hosting	Vercel, static. No backend.	There is nothing for us to run, which supports the no-operator argument.
Seed	Script: synthetic shops, two zones, a verifier, three runners with history	Judges open the link and see a live board.

Core calls (to verify against the 0.8.x API reference on day one)

```
import { createPublicClient, createWalletClient, ExpirationTime, jsonToPayload }
  from "@arkiv-network/sdk"
import { tiramisu } from "@arkiv-network/sdk/chains"
import { str, u64, i32, addr, key } from "@arkiv-network/sdk/attr"

// Runner announces a run
await runner.createEntity({
  payload: jsonToPayload({ note: "via 80 ft road" }), contentType: "application/json",
  attributes: { project: str("bhaiyaplease"), type: str("run"), dest_zone: str("tdr1wx"),
                from_zone: str("tdr1wq"), eta: u64(etaUnix), capacity_left: i32(2), tier: i32(1) },
  expires: ExpirationTime.fromSeconds((etaMin + 10) * 60),
})

// Resident issues a read-only receipt and hands it to the runner in one atomic batch
const [r] = await publicClient.predictEntityKeys({ owner: resident.account.address, count: 1 })
await resident.executeBatch({
  creates: [{
    payload: jsonToPayload({ tags: ["exact item", "on time"] }), contentType: "application/json",
    attributes: { project: str("bhaiyaplease"), type: str("receipt"), rider: addr(riderAddr),
                  delivery: key(deliveryKey), zone: str("tdr1wx"), at: u64(nowUnix) },
    expires: ExpirationTime.fromDays(365), flags: { readonly: true }, salt: r.salt,
  }],
  ownershipChanges: [{ entityKey: r.key, newOwner: riderAddr }],
})

// Shops: anyone may extend
await lister.createEntity({ /* ... */ expires: ExpirationTime.fromDays(30),
  flags: { permissionlessExtension: true } })
await anyNeighbour.extendEntity({ entityKey: shopKey, expires: ExpirationTime.fromDays(30) })
```

10. Build plan for two people (16 to 18 October)

> **Deadline:** Saturday 18 October, 23:59 London time = **04:29 IST on Sunday 19 October**. Submit by Saturday evening IST. The judged commit is the repo SHA at the deadline, and history must not be rewritten afterwards.

When	Person A: Arkiv and data	Person B: app and UX
Thu 16 evening	MIT repo, faucet, access key, SDK hello-world. Start friction.md. Typed entity helpers.	PWA shell, burner wallet with QR export, geohash, switch between runner and resident.
Fri 17 morning	Run, Request and Accept (batch with capacity patch). Watchers. Shop seed with permissionless extension.	Runner screens (announce, incoming requests) and resident screens (live runs, shop picker, request form).
Fri 17 evening	Delivery code reveal, then Receipt create plus transfer batch. Complaint. Attestation and verifier page.	Handoff screen, profile (attestations, distinct-resident count, complaints), tier badge on run cards.
Sat 18 morning	Final schema.md and friction.md. Collect wallet addresses. Optional atBlock dispute view.	Polish and deploy. Test on two real phones. **Interview 3 to 5 riders** outside a dark store.
Sat 18 afternoon	README with reproducible setup, and answers for the form.	Record the 3-minute demo.
Sat 18 evening IST	**Submit.** Final check against the gates list.	Keep the deployment up through 22 Oct.

Must ship	Cut if short on time
• Run, Request, Accept, Delivery and Receipt flow with live updates • Receipt transfer to the runner (the core Arkiv story) • Attestation and verifier page • Complaint, tier badge, profile • Shop seed with permissionless extension • schema.md, friction.md, demo video	• atBlock dispute view (keep it in schema.md) • Successor attestation for key loss • Map view (a list is enough) • No-show complaints against residents • Hindi or Kannada interface strings

11. Demo video script (3 minutes or less)

Time	Shot
0:00 to 0:20	The problem, plus a 10-second clip of a real rider interview.
0:20 to 0:45	The verifier scans the runner's QR code and issues an attestation. The runner's card shows "Verified".
0:45 to 1:20	The runner announces a run. The resident sees it live, picks a kirana on the route and requests curd. The runner accepts.
1:20 to 1:50	At the gate the runner scans the handoff QR code, the delivery is verified, and the receipt is issued. The runner's count goes up.
1:50 to 2:25	Explorer: the receipt's $creator is the resident and $owner is the runner, and it is read-only. A complaint is owned by the resident. The run vanishes after expiry.
2:25 to 3:00	"On Firebase, reputation is a feature of our app. On Arkiv, it's an asset of the runner." Close with "Bhaiya, please le aana if possible."

12. Submission gates checklist

Gate	Plan
Public deployment, no access request	Vercel URL. Burner wallets mean judges can use it instantly.
Reproducible setup	README: clone, `npm i`, put the RPC and access key in .env, `npm run seed`, `npm run dev`.
Public repo, MIT licence	LICENSE in the first commit.
/arkiv/schema.md and /arkiv/friction.md	Section 5, finalised. Friction logged live.
Wallet addresses on Tiramisu	Seed, verifier and demo wallets, listed in the README.
Demo video of 3 minutes or less	Section 11.
Built 9 to 18 Oct, synthetic data only	Fresh repo. Fake shops and runners. No real names, numbers or addresses.
At least one Indian national on the team	Declared at registration and on the submission form.

13. Beyond the hackathon

- **Society integration:** resident welfare associations act as verifiers, and gate apps read runner records directly from Arkiv.
- **Runner-owned credentials:** a runner shows a verifiable history to a lender, a new platform or a welfare board, building on the platform-worker laws in Karnataka and Rajasthan.
- **Open dataset:** the community-maintained, self-expiring shop map becomes a public record of India's informal retail.